Problem Statement & Objectives

Scalable Network I/O: From select to io_uring | CS331 | Team T016 | Project ID 13

Problem

- One thread serves many clients, so it must ask the kernel which sockets are ready
- Linux has 4 ways to ask. Which one should a server use?
- Does the answer change with the application (small messages, file transfer, streaming), the number of idle clients and the load?

Objectives

- Framework first, applications as plugins (TCP and UDP)
- 4 engines containing only their event loop
- Noop baseline + echo, upload, download, stream, memory
- Vary connections (1-1000), idle clients, size, load, threads
- Explain results with syscalls, wakeups, interrupts, CPU, memory

The four mechanisms

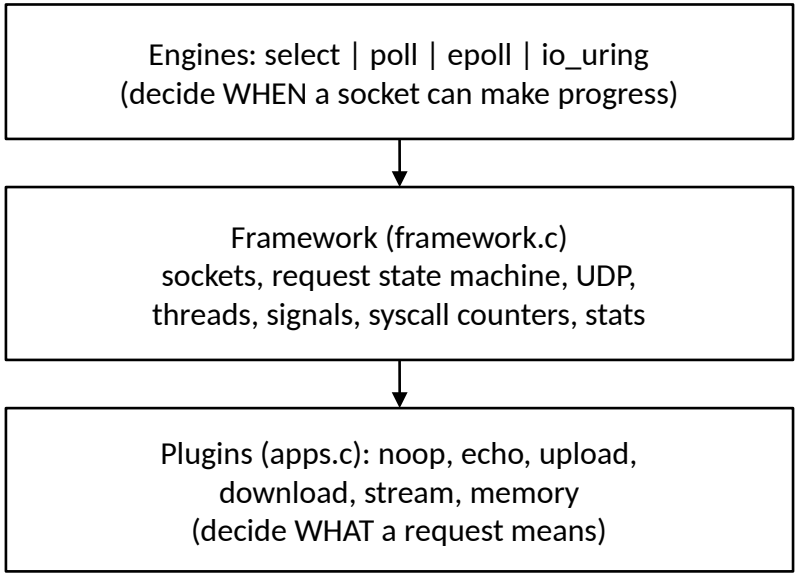
Mechanism	Type	Cost of one call
select (1983)	Readiness	Copies + scans ALL fds; max 1024 fds
poll (1986)	Readiness	Scans ALL fds; no 1024 limit
epoll (2002)	Readiness	Returns only READY fds; epoll_ctl to change interest
io_uring (2019)	Completion	Shared rings; many operations per syscall (or none)

Readiness: kernel says "socket ready", then we call recv/send.
Completion: we hand the kernel recv/send, it tells us when done.

What earlier papers report

- All connections busy: select/poll close to epoll (Gammo 2004)
- Many idle connections: select/poll slow down, epoll does not (Banga 1999)
- io_uring: modest gain from batching syscalls (arXiv 2025)

Architecture / Mechanism



Plugin	Represents	Request -> reply
noop	Baseline (no work)	8 B header -> 1 B
echo	Chat (latency)	64 B -> same 64 B
upload / download	File transfer	1 MiB in / 1 MiB out
stream	Video frames	64 KiB, 30 per second
memory	Memory-heavy	256 KiB per client

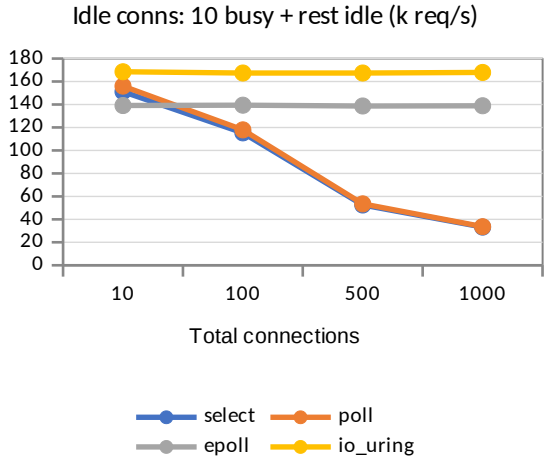
How it works

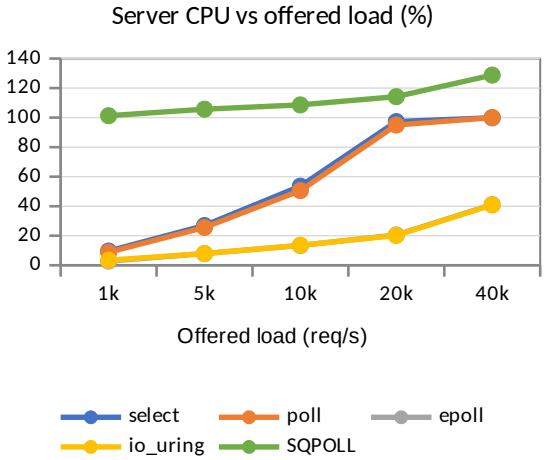
- Request = 8-byte length header + body; partial reads/writes handled once, for all engines
- select / poll / epoll: wait until a socket is ready, then recv / send
- io_uring: submit recv / send to a ring; one syscall can carry many operations
- Framework and plugins compiled once and linked into all 4 servers, so only the engine differs
- C load generator over loopback: closed loop (max speed) or paced (fixed rate); 2 s warm-up + 10 s measured

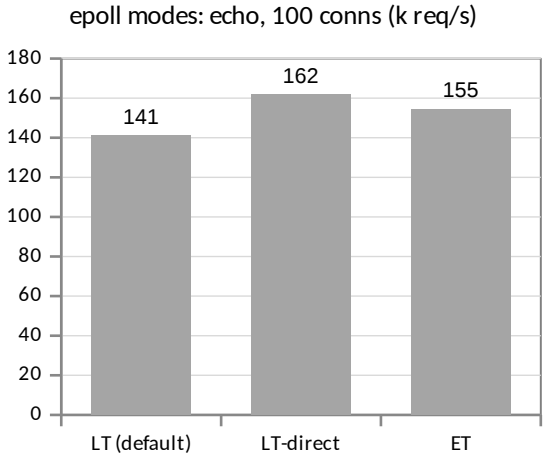
Extensions (built on top)

- epoll: level-triggered (LT), edge-triggered (ET), LT-direct (reply at once)
- io_uring: SQPOLL (kernel thread submits) and SQPOLL + busy polling
- 1, 2, 4 event loops with SO_REUSEPORT (no locks)
- UDP mode in all 4 engines (64 B to 4 KiB datagrams)
- Paced rate sweep, 1k to 40k req/s, for CPU and latency at a fixed load

Evaluation (1/2): Connections, Load, epoll Modes

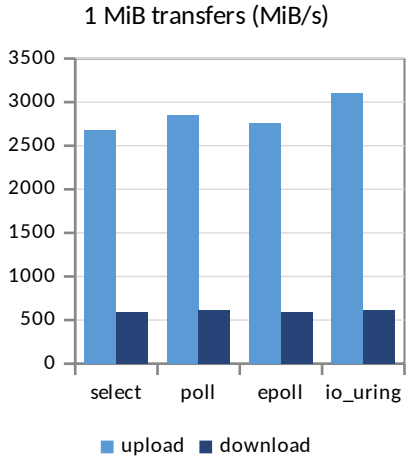


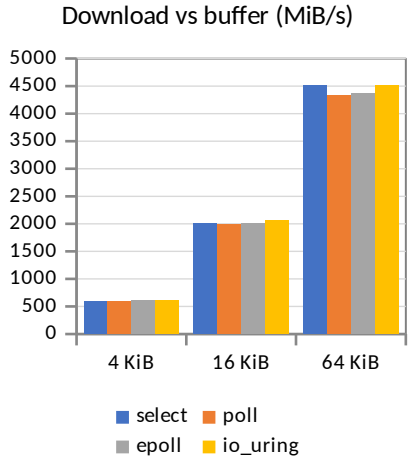


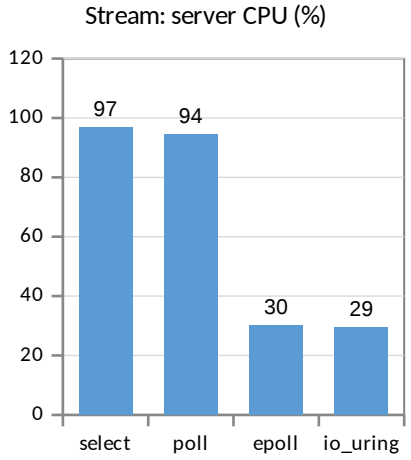


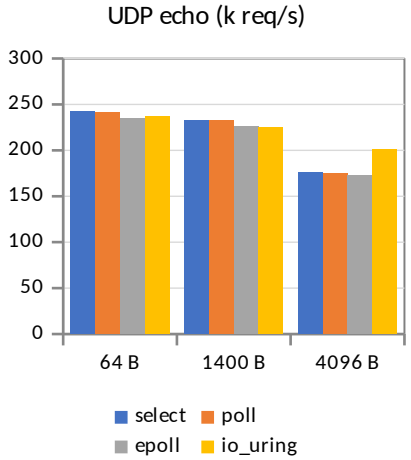
- 1000 connections (990 idle): select/poll drop 78% (151k -> 33k req/s) because they check every fd on each call; epoll and io_uring stay flat
- All 100 clients busy: io_uring fastest (176k vs 165k req/s), with 0.03 kernel entries per request
- Default epoll slowest (141k): 2 extra epoll_ctl calls per request. LT-direct and ET avoid them; ET also downloads 14% faster
- Plain io_uring CPU follows the load like epoll (3% -> 41%)
- SQPOLL stays above 100% even at 1k req/s: its kernel thread keeps polling; at full load it is no faster
- Light load: select/poll need 4x the CPU and 3x the latency (54 vs 18 us), since each wakeup finds ~1 ready socket but scans all 100
- 1 connection: all engines 64-67k req/s, nothing to batch

Evaluation (2/2): Transfers, Streaming, UDP







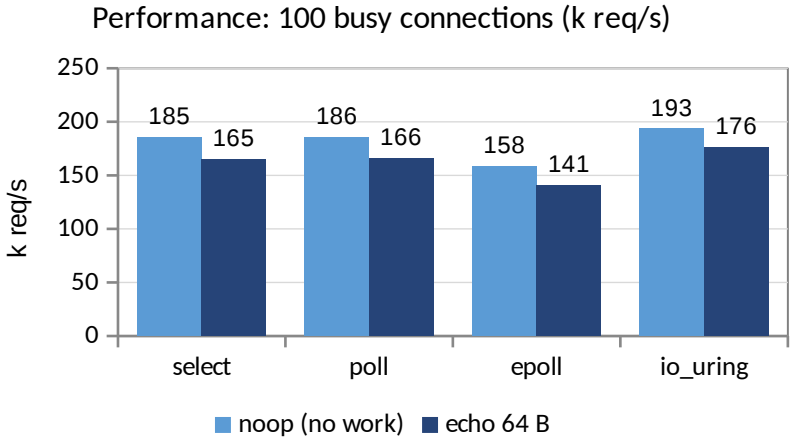


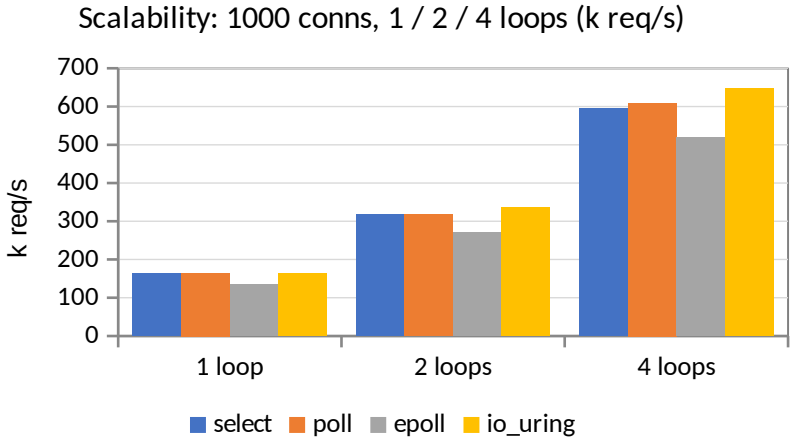
- Upload: io_uring 9-15% faster, 27 instead of ~292 syscalls per MiB
- Download costs 4.6x the CPU of upload: on loopback the sending server also does the receiver's network work
- Buffer size matters more than the engine: 4 -> 64 KiB gives ~7.5x faster downloads for all engines (16x more data per syscall)

- Streaming (30 frames/s): select/poll need 3.2x the CPU and 3x the latency, since each frame needs ~17 waits that scan all 100 fds
- UDP: one socket for all clients, so no scan; select/poll as fast as epoll. io_uring (32 receives queued) 14% faster at 4 KiB
- Memory-heavy (500 x 256 KiB): 130 MiB for every engine vs 1.8 MiB for echo; memory comes from the app, not the engine

Non-Functional Testing Parameters

Parameter	What we measured / checked
Performance	Throughput, p50/p99 latency, CPU time per request, syscalls per message
Scalability	1 to 1000 connections, up to 990 idle; 1/2/4 event loops (near-linear)
Reliability	159/159 correctness tests (byte-verified data, resets, signals, 1000 clients); all 1140 runs valid
Accuracy	In-server syscall counters match strace; C client matches an independent Python client
Repeatability	5 runs per setting, all runs shuffled; median spread between runs 1.3%
Resources	CPU, memory (1.8 MiB echo, 130 MiB for 500 memory clients), interrupts, wakeups
Fairness	Same app code in all engines; client never the bottleneck (max 73% CPU per thread)





Challenges Faced

Challenge	How we handled it
Fair comparison of 4 servers	Shared framework and app code; only the event loop differs
strace slows the server	Server counts its own syscalls; matches strace exactly
Waiting client hides slow servers	Paced mode: latency from the scheduled send time
Loopback: receiver's work on sender's CPU	Softirqs counted separately
Rare latency spikes	10 ms sampling: other programs briefly on the server CPU
Old liburing 2.1	No multishot receive; left for future work
5-hour run on one laptop	Runs shuffled, 5 repeats, so drift cannot favour an engine

Issues found and fixed

- Each server had its own copy of the app code; the compiler optimised it differently (2 engines looked 60% faster). Now one shared copy
- io_uring completion queue overflowed at 1000 clients
- io_uring stalled when the submission queue was full
- UDP drops: 8 MiB buffers + more pending receives
- Paced client threads sent in bursts: start times staggered

Future work

- Two machines with a real network card instead of loopback
- Newer io_uring features: multishot receive, registered buffers, zero-copy send
- Compare with thread-per-connection servers